gomesf@chalmers.se
Computer Science and Engineering Department
Chalmers and the University of Gothenburg

DAT321 | DIT843 External Quality

Dr. Francisco Gomes de Oliveira Neto

Grace Hopper

- Pioneer in High-Level Programming Languages.
- Coined the term 'bug' for describing computing errors.
- One of the first programmers of the Harvard Mark I computer.

What will we learn today?

- How to create **quality scenarios**?
- How to measure **external quality**?
- What are the different **metrics** for **reliability**?
- What are the different **metrics** for **performance efficiency**?

Part 1:
Quality Scenarios

© Francisco Oliveira Neto - 2018

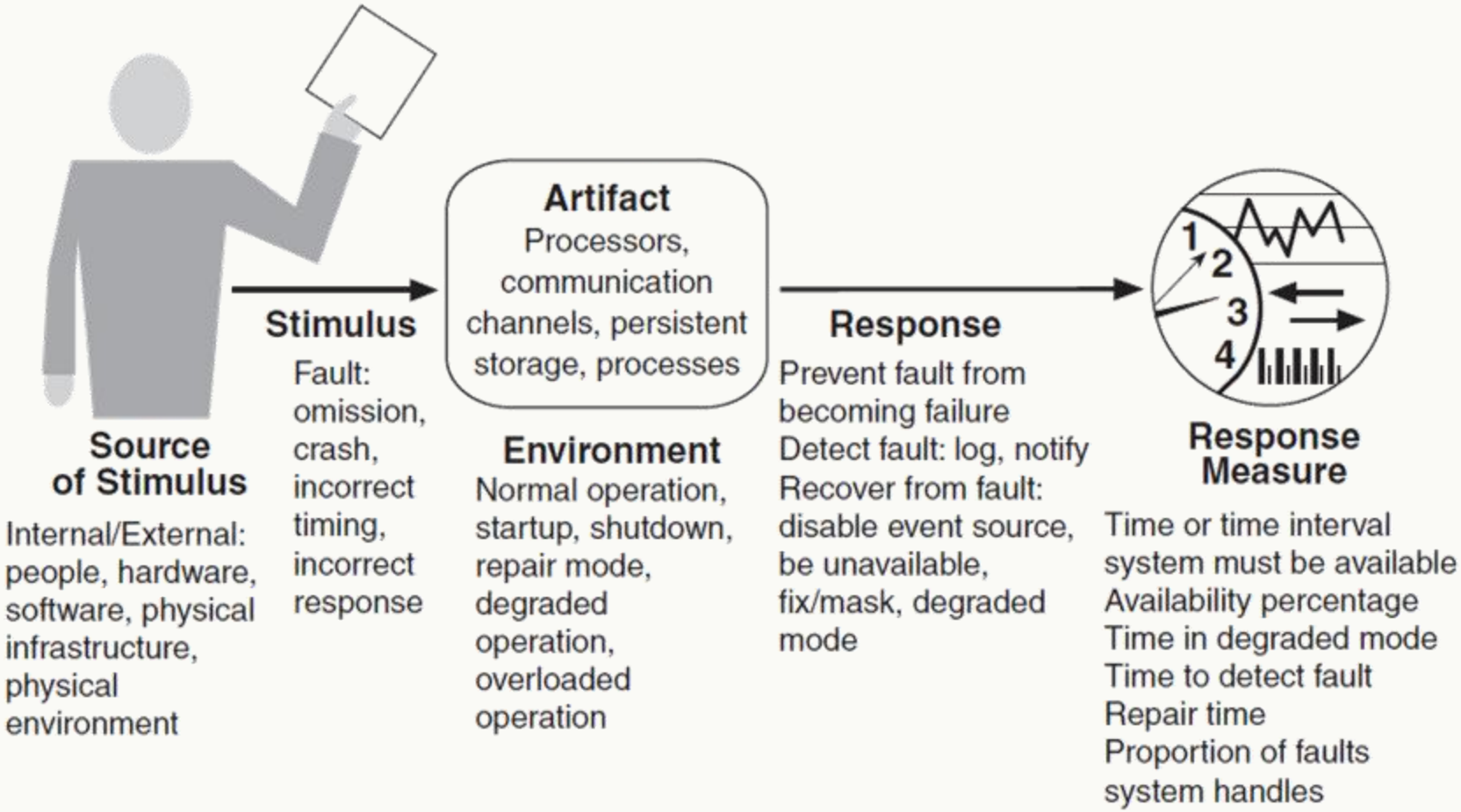
Quality Scenarios – Six elements

1. **Source** of stimulus. This is some entity (a human, a system, etc.) that generated the stimulus.
2. **Stimulus**. The stimulus is an action that requires a response when it arrives at a system.
3. **Environment**. The stimulus occurs under certain conditions. (e.g., overloaded, service unavailable, etc.)
4. **Artifact**. A deliverable created in the development process (code, design document, build log, etc.)
5. **Response**. The response is the activity undertaken as the result of the arrival of the stimulus.
6. **Response measure**. A value (quantitative or qualitative) associated to the response.

- One of the main benefits is that a good quality scenario provides: **test cases and acceptance criteria**.

© Francisco Oliveira Neto - 2020

Quality Scenarios – Six elements



Src: Bass et al., Software Architecture in Practice Third Edition. (Chapter 4, page 70) 2015

© Francisco Oliveira Neto - 2020

Example 1 – ChatApp

You work for the Chat-Co, a company that offers a software product for chatting or messaging friends (think of WhatsApp, Messenger). Below we list important features about the software product and its development process:

F1: Each user account is connected to a single phone number. This number registered when creating the account.

F2: Besides text messages, users can send audio or video clips (limit of 3 minutes per message), or image files.

F3: Users can make video or audio calls with another user or a group chat that they are a part of. The user(s) on the other side of the call must accept the call before initiating the audio / video stream.

F4: The ChatApp can be used in mobile phones, tablets, or personal computers (PCs).

F5: The user interface must adapt, i.e., be responsive to the different layouts of screens from multiple types of devices.

F6: The development team does not have a fixed release calendar. New features and bug fixes are only added based on requests by the community of users.

F7: Users can customize their user interface by choosing different color themes, backgrounds, fonts, and text color in their chats.

F8: The ChatApp can import a list of contacts from the user's personal accounts (Google, Apple, Facebook, etc.).

Example 1 - ChatApp

- For each of the quality attribute chosen for ChatApp, create a quality scenario for the game that includes: (i) source, (ii) stimulus, (iii) artefact, (iv) environment, (v) response, and (vi) response measure.

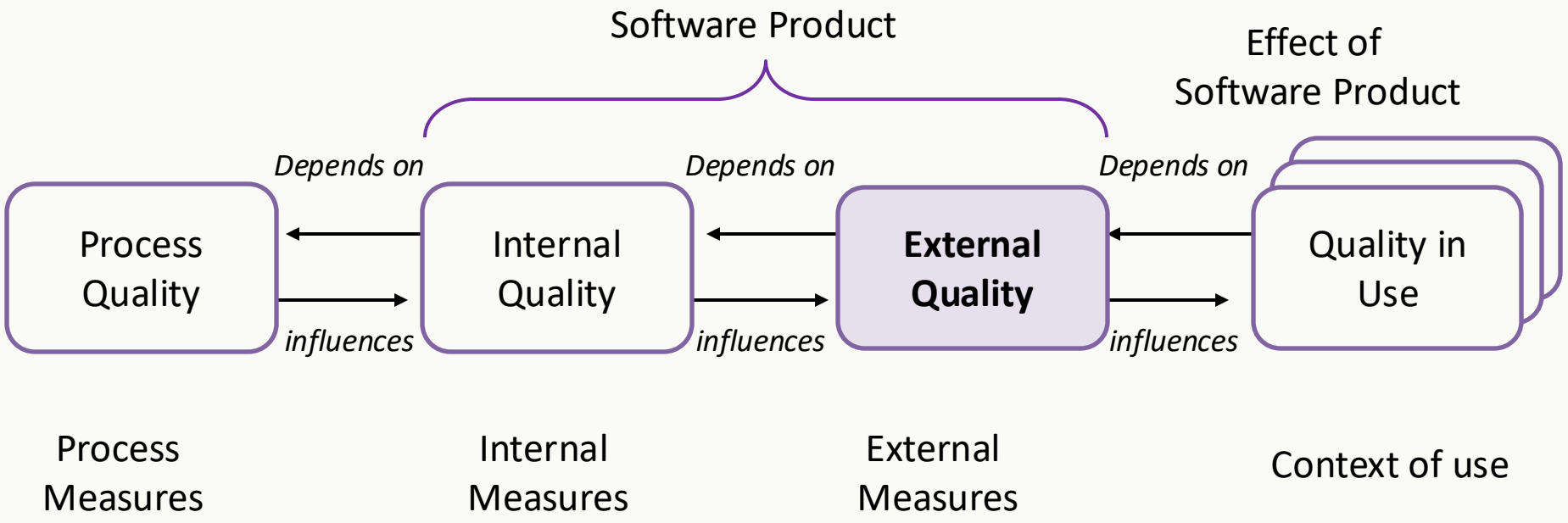
Part 2:
External Quality

© Francisco Oliveira Neto - 2018

External Quality

- Metrics related to **software execution**.
- These measures are relevant to Continuous Delivery activity, and other tasks that involve **monitoring**, **logging**, **benchmarking** or **profiling** the software system.
- We will focus on measures for **Reliability** and **Performance Efficiency**.

External Quality



Source: International Standard Organization, ISO/IEC 25010 standard

Reliability

Reliability

- Degree to which a system, product or component performs **specified functions** under **specified conditions** for a **specified period of time**.

- We improve reliability by **removing faults**. Not all faults impact reliability the same way.
  - For instance, fixing faults in the more frequently used parts has a higher impact in reliability.

Reliability

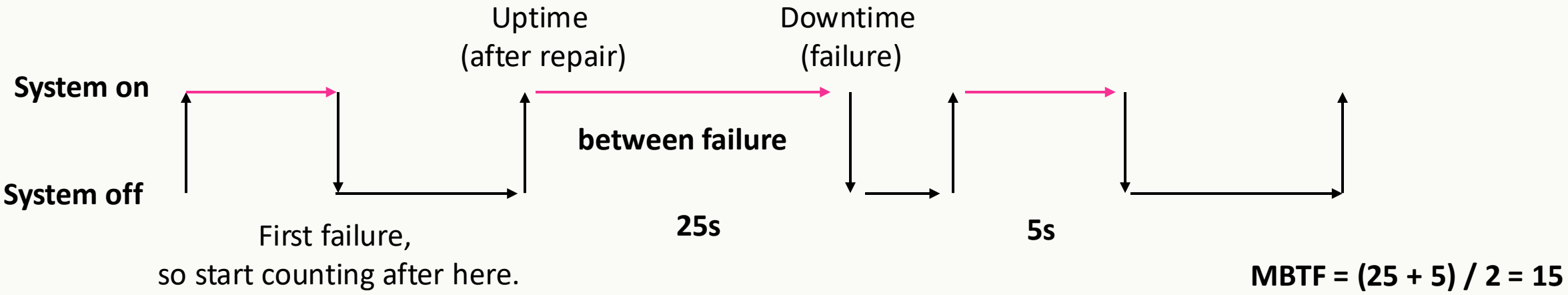
- We cannot measure reliability without a **context**.
- **Important Tip:** Find **thresholds** acceptable for the chosen context.
  - What is the specified **function**?
  - What are the specified **condition**?
  - What is the specified **period of time**?
- Popular reliability **metrics**: Failure rate, uptime, time between failures, etc.

Reliability - Failure Rate Measures

- **Probability** of Failure on Demand (**POFOD**) = **failed** requests / **total** requests
- **Rate** of Occurrence of Fault (**ROCOF**) = **failed** requests **/ time**
- **Independent of frequency or time**. Example: 100 requests, 3 of them failed: POFOD = 0.03.
  - Case 1: 100 requests in 0.1 seconds (e.g., shopping via Amazon)
  - Case 2: 100 requests in 10 seconds (e.g., booking Covid-19 vaccine)
  - Case 3: 100 requests in 1 month (e.g., request your location on a GPS system)
  - All cases have the same POFOD, but different impact with different levels of severity (ROCOF).
- Remember that reliability **depends on a specified context**, function and time.

Mean Time Between Failures (MTBF)

- Expected **time** until a failure occurs. Mean is **sensitive to outliers**.
- MBTF = $\frac{\sum failure_time_i - uptime_i}{num_failures}$

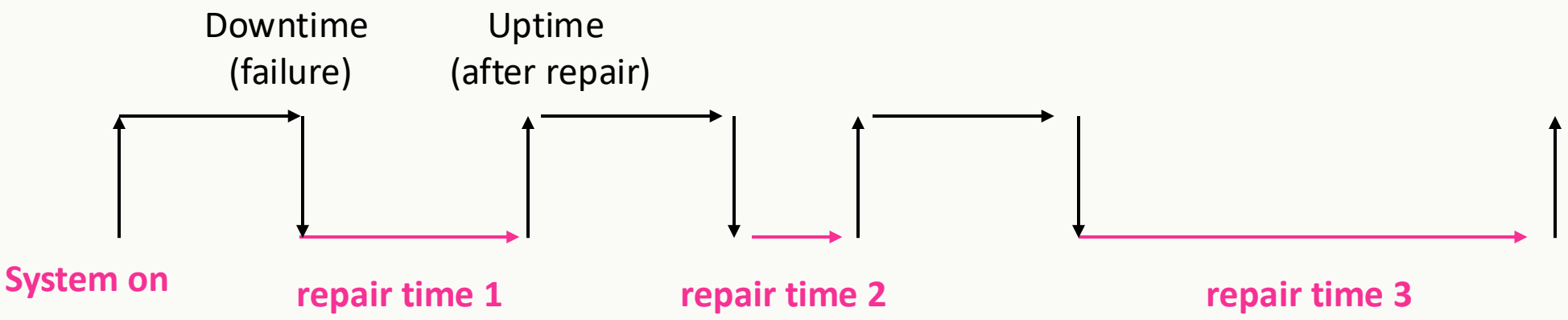


Mean Time Between Failures (MTBF)

- MBTF = $\frac{\sum(failure_time_i - uptime_i)}{num_failures}$ (using the timestamps)
- Examples:
  - How often should PowerPoint **automatically save your work**?
    - Saving every second would make it slow, but finding out the MTBF from typical usage, can help developers define that saving every, e.g., 5 minutes is good enough.
  - When should we **roll back a new version** released to a subset of users?
    - MTBF can be used as a threasholf in canary releases (feature release to subset of users).

Mean Time to Repair (MTTR)

- Same as MBTF but using the **repair time instead**.
- Correlating MBTF and MTTR helps us understand the rate of **availability** of our system.



Availability

- Availability = MTBF / (MTBF + MTTR).
- Simpler version: Availability = **Uptime** / **Total time** observed
  - A simpler version of availability is the percentage of uptime of a software (e.g., service). **Only consider crashes** (even if there are failures).
- What do you believe is an **acceptable availability**? For instance,
  - Is there a difference in availability for 80%, 90%, 95%, 96%, 99%, 99.5%?

Availability %	Downtime per day	Downtime per week	Downtime per month	Downtime per year
90% ("one nine")	2.40 hours	16.80 hours	73.05 hours	36.53 days
95% ("one and a half nines")	1.20 hours	8.40 hours	36.53 hours	18.26 days
97%	43.20 minutes	5.04 hours	21.92 hours	10.96 days
98%	28.80 minutes	3.36 hours	14.61 hours	7.31 days
99% ("two nines")	14.40 minutes	1.68 hours	7.31 hours	3.65 days
99.5% ("two and a half nines")	7.20 minutes	50.40 minutes	3.65 hours	1.83 days
99.9% ("three nines")	1.44 minutes	10.08 minutes	43.83 minutes	8.77 hours
99.995% ("four and a half nines")	4.32 seconds	30.24 seconds	2.19 minutes	26.30 minutes
99.999% ("five nines")	864.00 milliseconds	6.05 seconds	26.30 seconds	5.26 minutes
99.9999% ("six nines")	86.40 milliseconds	604.80 milliseconds	2.63 seconds	31.56 seconds
99.99999% ("seven nines")	8.64 milliseconds	60.48 milliseconds	262.98 milliseconds	3.16 seconds
99.999999% ("eight nines")	864.00 microseconds	6.05 milliseconds	26.30 milliseconds	315.58 milliseconds
99.9999999% ("nine nines")	86.40 microseconds	604.80 microseconds	2.63 milliseconds	31.56 milliseconds

Availability %	Downtime per day	Downtime per week	Downtime per month	Downtime per year
90% ("one nine")	2.40 hours	16.80 hours	73.05 hours	36.53 days
95% ("one and a half nines")	1.20 hours	8.40 hours	36.53 hours	18.26 days
97%	43			
98%	2			
99% ("two nines")	1			
99.5% ("two and a half nines")	7			
99.9% ("three nines")	1			
99.995% ("four and a half nines")	4			
99.999% ("five nines")	8			
99.9999% ("six nines")	86.40 milliseconds	604.80 milliseconds	2.63 seconds	31.56 seconds
99.99999% ("seven nines")	8.64 milliseconds	60.48 milliseconds	262.98 milliseconds	3.16 seconds
99.999999% ("eight nines")	864.00 microseconds	6.05 milliseconds	26.30 milliseconds	315.58 milliseconds
99.9999999% ("nine nines")	86.40 microseconds	604.80 microseconds	2.63 milliseconds	31.56 milliseconds

Terminology of "9 nines". Refers to different thresholds of availability. Compare **how the downtime grows** per month or year. Can vary depending if we are talking about hardrware or software. Remember to **consider the context description**.

Summary:

- **POFOD**: Failed Requests / Total Requests
- **ROCOF**: Failed Requests / Time to send all requests
- **MTBF**: Average time between failures
- **MTTR**: Average time to repair the system (bring it back up)
- **Availability**: MTBF / (MTBF + MTTR) or (Uptime / Total Time)

Example 2 – Klarna example

- Imagine that you were hired as QA engineer in a company for online purchases (e.g., Klarna). You must evaluate the **reliability of the purchase** requests for a daily usage in the conditions below.

  - **Q1- How do you characterise your reliability analysis?**
  - Q2- During an entire day, you measure 50000 requests, 30 of them failed (all of them are crashes). **What is your POFOD and your ROCOF?**
  - Q3- Weekend purchase is higher, where we get 500000 requests per day. **How many failed purchases should we expect?** (Weekend = 2 days)

Example 2 – Klarna purchase

- Q1- Reliability framing:
  - Specific function: Purchase requests.
  - Specific condition: Regular Weekday (in q2); Regular Weekend (in q3)
  - Specific time: Day

- Q2- POFOD = 30 / 50000 = 0.0006 (0.06 %)
- Q2- ROCOF = 30 / 24 hours = 1.25 failures per hour.
- Q3- 0.0006 * 500000 = 300 * 2 days = 600 failures over the weekend

Example 3 – Klarna example

- Imagine that **every time that there is a failure**, it takes 5 minutes to bring the system up again. **What is Klarna's availability per day?**

- Your boss asks whether we need to improve availability of our services. **If not needed, explain why. If needed, how can we improve it?**

Example 3 – Klarna availability

- 1 Day == 1440 minutes. 30 failures per day (see prev. slides)
- 5 minutes each crash = 5 * 30 = 150 min downtime per day.
- Availability = (1440 – 150) / 1440

  = 0.895 availability per day (really low!)

- What should we do? Try to decrease downtime or POFOD.

Reliability trade-off

- Sometimes fixing bugs or preventing **crashes can be expensive**. It may be **cheaper to accept a liability**.
- Highly dependant on the **context**. No right answer.
  - What could be the cost of a failure? Money, time, a human life?

- Understanding failure rate and availability **allows you to predict**, plan and decide how to ensure **dependability of your software**.

Programming Guidelines for Reliability

1. **Limit the visibility** of information in a program
2. Check all inputs for **validity**
3. Provide a **handler** for all exceptions
4. **Minimize** the use of error-prone constructs
5. Provide **restart** capabilities
6. Check **array bounds**
7. **Include timeouts** when calling external components
8. Name all **constants** that represent real-world values

Performance Efficiency

Performance Efficiency

- Related to the **resources** used under stated conditions. Typically, it relates to time or memory.
- **Popular** Metrics / Issues: Latency and Throughput
- Impacts the **scalability** of the system.
- **Performance requirements** may vary depending on the context (even if not explicitly stated).

Latency and Jitter

- **Latency** = reaction_timestamp – request_timestamp
- Latency is often related to communication, but it is a pervasive **measure of responsiveness**. For instance, responsiveness of a (gaming) controller, communication with a sensor, or simply reacting to a end-user's action.
- Sensisitive to variations. **Jittering conveys the variation** in lattency. Useful to define a window of time performance.
  - Ex: Latency of 300 ms with + / - 10 ms.

Throughouput

- Tasks completed (**workload**) over a period of time.
- Related to latency, but focused on **completion of tasks**, rather than individual response time.
- Throughput and response time **affect one another**. If we have more requests, and the same resources, each processing time takes longer, hence reducing throughput.
- For instance, we measured:
  - 100 concurrent users. Each user waits 1 minute to complete: 100 completion / min
  - 1000 concurrent users. Each user waits takes 2 minutes to complete: 500 completion / min
  - Note, we have **higher throughput but also higher response time** (latency).

Example – Developer's Tools

- We can use the terminal and ping to measure latency of web domains. Try: `ping www.chalmers.se` (Use CTRL + C to interrupt the process.)

- **Browsers have developer's tools** with information about performance measures:
  - Exercise: Measure and compare latency in accessing the webpages below.
  - Calculate both latency and jittering
    - Chalmers: https://www.chalmers.se/en/research/
    - GU: https://www.gu.se/forskning
    - KTH: https://www.kth.se/en/forskning